

Full Stack Developer — Step 6

Full Stack Projects

Roadmap objective: Build an integrated frontend, backend and database application

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Architecture
- Feature slicing
- API contract
- Database schema
- Authentication
- Error states
- Testing
- Git workflow
- Deployment
- README

Core concepts

1. Build vertically: complete one feature from UI to API to database before adding many disconnected screens.
2. Keep API contracts explicit so frontend and backend can evolve independently.
3. A good project demonstrates decisions and trade-offs, not just a large number of pages.
4. Deployment should be treated as part of development, not a final afterthought.

Concept map

```
Full Stack Projects
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Architecture

Study architecture through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Feature slicing

Study feature slicing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. API contract

Study api contract through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Database schema

Study database schema through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Authentication

Study authentication through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Error states

Study error states through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Feature slice

Feature: Create a roadmap step

```
UI form
-> POST /api/roadmap-steps
-> validate request
-> insert into MySQL
-> return 201 + created object
-> update UI
-> show success/error state
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Monolith	One deployable application	Simple start
Service-based	Separated services	More operational complexity
Vertical slice	UI→API→DB feature	Fast feedback

Common mistakes

- Building all UI before backend
- Skipping API contracts
- No error/loading states
- No deployment until the end
- No tests for important flows

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Choose one feature and build it end-to-end.
- Write down what you expected, what happened, and why.

Lab 2

- Write the API contract.
- Write down what you expected, what happened, and why.

Lab 3

- Create the DB table/collection.
- Write down what you expected, what happened, and why.

Lab 4

- Implement backend logic.
- Write down what you expected, what happened, and why.

Lab 5

- Connect the frontend.
- Write down what you expected, what happened, and why.

Lab 6

- Deploy and test the same feature in production.
- Write down what you expected, what happened, and why.

Mini-project

Build a full-stack Career Tracker with login, career list, roadmap progress, database persistence, API validation, error states, Git history and deployment.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Why build vertical slices?

- Answer structure: definition → example → trade-off/limitation → practical use.

What belongs in an API contract?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you test a full-stack feature?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you handle errors in the UI?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why deploy before the end?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you explain trade-offs in a project?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Vertical slice	Write this in your own words.	Give one project example.
Architecture	Write this in your own words.	Give one project example.
Contract	Write this in your own words.	Give one project example.
Persistence	Write this in your own words.	Give one project example.
Authentication	Write this in your own words.	Give one project example.
Integration test	Write this in your own words.	Give one project example.
Deployment	Write this in your own words.	Give one project example.
CI/CD	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Error state	Write this in your own words.	Give one project example.
Trade-off	Write this in your own words.	Give one project example.

Self-test

- Explain Architecture without using its textbook definition.
- Show a small example of Feature slicing.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Architecture.
- I can explain and demonstrate Feature slicing.
- I can explain and demonstrate API contract.
- I can explain and demonstrate Database schema.
- I can explain and demonstrate Authentication.
- I can explain and demonstrate Error states.
- I can explain and demonstrate Testing.
- I can explain and demonstrate Git workflow.
- I can explain and demonstrate Deployment.
- I can explain and demonstrate README.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://docs.github.com/>
- <https://render.com/docs>
- <https://vercel.com/docs>

Recommended videos

- Full Stack Project Tutorial (Existing DB resource 115)
- Docker Tutorial - freeCodeCamp (Existing DB resource 105)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.